

```

9      0 referências
10     ✓ public class KissService
11     {
12         //Exemplo - Sem KISS(Errado)
13         0 referências
14         public int CalculateSumOfEvenNumbers(int[] numbers)
15         {
16             int sum = 0;
17             for (int i = 0; i < numbers.Length; i++)
18             {
19                 if (numbers[i] % 2 == 0)
20                 {
21                     sum += numbers[i];
22                 }
23             }
24             return sum;
25         }
26     }

```

Embora o código funcione, ele é mais complexo do que precisa ser. Podemos simplificá-lo.

```

0 referências
✓ public class KissService
{
    //Exemplo - Com KISS(Certo)

    0 referências
    ✓ public int CalculateSumOfEvenNumbers(int[] numbers)
    {
        return numbers.Where(n => n % 2 == 0).Sum();
    }
}

```

Por que está certo?

O uso de LINQ simplifica a lógica.
Mais legível e direto ao ponto.

```

10 0 referências
11  public class KissService
12  {
13      //Exemplo 2 - Sem KISS(Errado)
14
15      0 referências
16      public string GetStatus(int statusCode)
17      {
18          if (statusCode == 1)
19          {
20              return "Success";
21          }
22          else if (statusCode == 2)
23          {
24              return "Error";
25          }
26          else if (statusCode == 3)
27          {
28              return "Pending";
29          }
30          else
31          {
32              return "Unknown";
33          }
34      }

```

Problemas:

Muitas condições tornam o código mais difícil de ler e manter.
É propenso a erros caso sejam adicionados novos códigos de status.

```

0 referências
public class KissService
{
    //Exemplo 2 - Com KISS(Certo)

    0 referências
    public string GetStatus(int statusCode)
    {
        return statusCode switch
        {
            1 => "Success",
            2 => "Error",
            3 => "Pending",
            _ => "Unknown"
        };
    }
}

```

Por que está certo?

O uso do switch torna o código mais compacto e fácil de entender.

Simplicidade na adição de novos casos no futuro.

Como aplicar KISS no dia a dia?

Evite over-engineering: Não adicione complexidade desnecessária.

Use padrões e ferramentas apropriadas: Aproveite recursos da linguagem para simplificar seu código (ex.: LINQ, switch, etc.).

Divida responsabilidades: Evite métodos ou classes que fazem "tudo".

Revisão constante: Sempre pergunte: "Isso poderia ser mais simples?"

Resumo

Soluções simples são frequentemente as mais eficazes. Aplique o KISS escrevendo código direto, que faça apenas o necessário e seja fácil de entender. Evite complicar! □